So, $\overset{\text{sentence 1}}{\text{orange}} = \dfrac{e^{1}\,\text{orange} + e^{0.71}\,\text{Apple}}{e^{1} + e^{0.71}} = 0.57\,\text{Orange} + 0.43\,\text{Apple}$

& $\overset{\text{new sentence}}{\text{Orange}} = \dfrac{e^{1}\,\text{orange} + e^{-1}\,\text{Motorcycle}}{e^{1} + e^{-1}}$

$= 0.88\,\text{orange} + 0.12\,\text{Motorcycle}$

↳ This step is called softmax.

So, For 1st sentence

	orange	Apple	And	An
orange	1	0.71	0	0
	↓	↓	↓	↓
	e^{1}	$e^{0.71}$	e^{0}	e^{0}

So, $\text{Orange} = \dfrac{e^{1}\,\text{orange} + e^{0.71}\,\text{Apple} + e^{0}\,\text{Apple} + e^{0}\,\text{An}}{e^{1} + e^{0.71} + e^{0} + e^{0}}$

$\text{Orange} = 0.4\,\text{Orange} + 0.3\,\text{Apple} + 0.15\,\text{And} + 0.15\,\text{An}.$

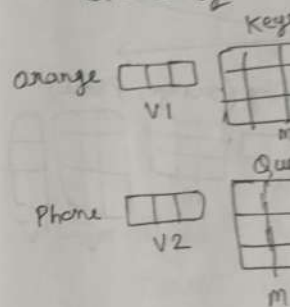
⇒ The key & Que

↳ These matrices h

where we can do atte

How to do it?

~~Similarity~~

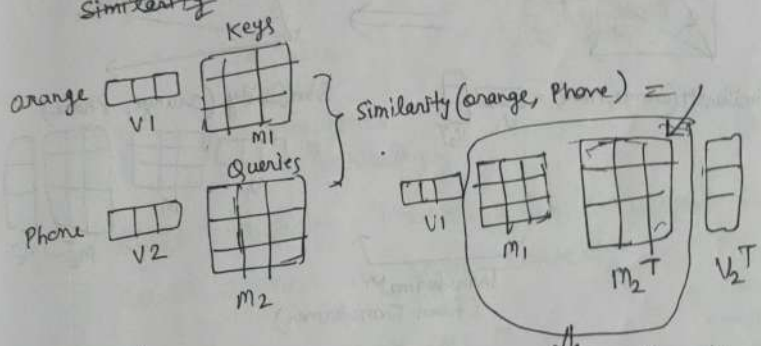


⇒ The Key & Query matrices

↳ These matrices help us to find really good embeddings where we can do attention & get a good infom. (okay, good, bad) from previous

How to do it?

~~Similarity~~



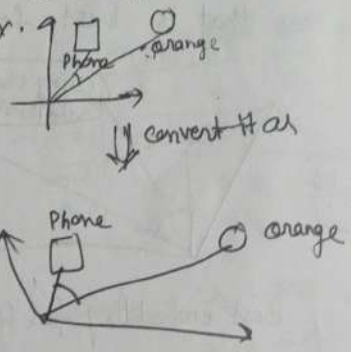
⇓

Linear Transformat^n

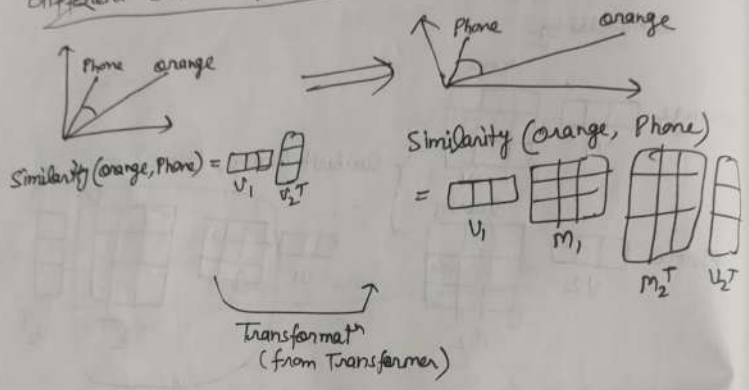
⇓

Convert one embedding to another,

Ex.

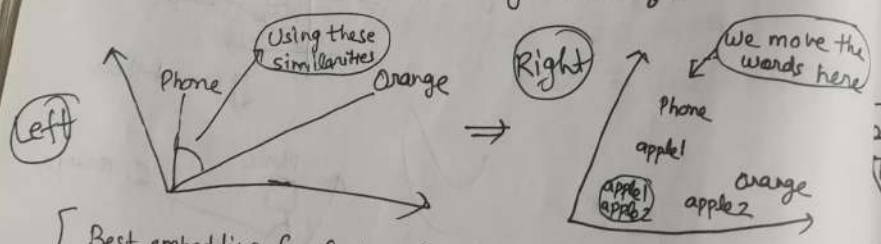


So, the keys & queries matrices work together to create a linear transformation that will improve our embedding to be able to do attention. So, we are modifying the similarity in one embedding to a similarity in one different embeddings & we are gonna make sure that this is best one



⇒ The Values Matrix

↳ Keys & Queries matrices actually turn the embedding into one that is best for calculating similarities.



[Best embedding for finding similarities using Keys & Queries Matrix]

[Best embeddings for finding the next word using Value Matrix]

What value

↳ It take
every vecto
[It consi

↳ Left emb

↳ Right
coul

⇒ Example

an apple a

	orange	Apple
orange	0.4	0.3
Apple	0.3	0.4
An	0.15	0.15
An	0.15	0.15

Apple = 0.3 orange
+0.4 Apple
+0.15 An
+0.15 A

What value matrix do?

↳ It takes the embedding on the left & multiplies every vector to get the embedding on the right.
[It considers as another transformatn]

↳ Left embedding(s) know the features of the words. (low & high level both)

- Color
- Size
- Fruitness
- Technology

↳ Right embedding knows when 2 ~~words~~ words could appear in same context.

I want to buy a ___

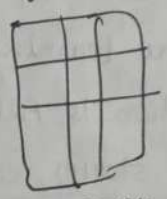
- car
- apple
- phone

↳ Example

an apple and an orange

	orange	Apple	And	An
orange	0.4	0.3	0.15	0.15
Apple	0.3	0.4	0.15	0.15
And	0.15	0.15	0.5	0.5
An	0.15	0.15	0.5	0.5

Value Matrix



(can be square or rectangle matrix)

=

	orange	Apple	And	An
orange	v_{11}	v_{12}	v_{13}	v_{14}
Apple	v_{21}	v_{22}	v_{23}	v_{24}
And	v_{31}	v_{32}	v_{33}	v_{34}
An	v_{41}	v_{42}	v_{43}	v_{44}

Apple = 0.3 orange
+0.4 Apple
+0.15 And
+0.15 An

Apple → v_{21} · orange
$+ v_{22}$ · apple
$+ v_{23}$ · and
$+ v_{24}$ · an.

⇒ In Multi-head attention we use many key, Query matrices
[The more we use the more likely we find better ones]

↳ We don't know which transformation is good & which is bad, so we concatenate them first (concat of 2 & 2 colums makes 4 columns)
[high-dimensional embedding]

& Now, linear step scales up better embedding & scales down bad embeddings by some amount.

↳ Finding Key (K), Query (Q) & Value (V) matrices are not easy.
They are trained with the transformer multiple times.

✱ What are Transformer Models & How do they work?

Transformer models are one of the most exciting new developments in Machine Learning. They were introduced in the paper "Attention is All you need.". Transformers can be used to write stories, essays, poems, answers questions, translate b/w languages, chat with humans, and they can even pass exams that are hard for humans.

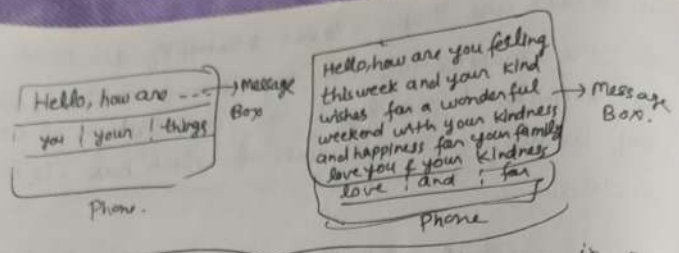
But what are
that the archi
that complex, i
very useful co
function.

↳ In a nutshe
Imagine th
your phone. Aft
suggested to y
the phone may
as the next
the suggested
find that the
no sense. I
consecutive u
don't concat
This is beca
carry the ov
predicts whic
the last few
track of t
is why t

But what are they? You'll be happy to know that the architecture of transformer models is not that complex, it simply is a concatenation of some very useful components, each of which has its own function.

↳ In a nutshell, what does a transformer do?

Imagine that you're writing a text message on your phone. After each word, you may get 3 words suggested to you. For example, if you type "Hello, howare", the phone may suggest words such as "you", or "your" as the next word. Of course, if you continue selecting the suggested word in your phone, you'll quickly find that the message formed by these words makes no sense. If you look at each set of 3 or 4 consecutive words, it may make sense, but these words don't concatenatenate to anything with a meaning. This is because the model used in the phone does not carry the overall context of the message, it simply predicts which word is more likely to come up after the last few. Transformers, on the other hand, keep track of the context of what is being written & this is why the text that they write makes sense.



The phone can suggest the next word to use in a text message, but does not have the power to generate coherent text.

↳ I have to be honest with you, the first time I found out that transformers build text one word at a time, I could not believe it. First of all, this is not how humans form sentences & thoughts. We first form a basic thought & then start refining it & adding words to it. This is also not how ML models do other things. For example, images are not built this way. Most neural network based graphical models form a rough version of the image & slowly refine it or add detail until it is perfect. So, why would a transformer model build text word by word? One answer is, because that works really well. A more satisfying one is that because transformers are so incredibly good at keeping track of the context, that the next word they pick is exactly what it needs to keep going with an idea.

And how are transformers trained? With a lot of data, all the data on the internet, in fact. So, when you input the sentence "Hello, how are" into the transformer, it simply knows that, based on all the text in the internet, the best next word is "you". If you were to give it a more complicated command, say, "Write a story", it may figure out that a good next word to use is "Once". Then it adds this word to the command, & figure out that a good next word is "upon" & so on. And word by word, it will continue until it writes a story

Command: Write a story

Response: Once

Next command: Write a story. Once

Response: upon

Next Command: Write a story. Once upon

Response: a

** Next Command: Write a story. Once upon a

** Response: ** time

** Next Command: Write a story. Once upon a time

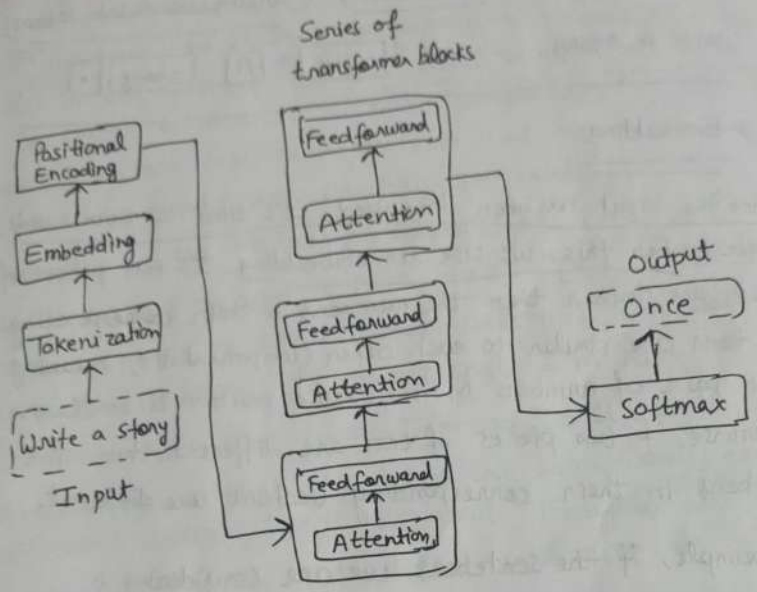
** Respone: ** there

etc.

Now that we know what transformers do, let's get to their architecture. If you've seen the architecture of a transformer model, you may have jumped in awe like I did the 1st time I saw it, it looks quite complicated. However, when you break it down into its most important parts, it's not so bad. The transformer has 4 main parts

- Tokenization
- Embedding
- Positional encoding
- Transformer block (several of these)
- Softmax

↳ The 4th one, the transformer block, is the most complex of all. Many of these can be concatenated & each one contains 2 main parts: The attention & the feedforward components.



The architecture of transformer model

Tokenization

Tokenization is the most basic step. ~~step~~ It consists of a large dataset of tokens, including all the words, punctuation signs etc. The tokenization step takes every word, prefix, suffix & punctuation signs & sends them to a known token from the library.